

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – High (Scenario Based Assessment)**

Course Code:	CSA0901	Course Title:	Programming in Java
CO Assessed:	CO2 – Precision (BL3,BL4)	Assessment:	ASSESSMENT TOOL 1 - High (Scenario Based Assessment)
Weightage:		Total Marks:	100
CO1 Statement:	Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)		
Student Name:	Pranav Adarsh G	Reg. No.:	1924260042
Section Batch: /	SLOT B	Date:	31/07/2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 90 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANNEXURE

Scenario Based Assessment

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

(a) Enumerate three Collection interfaces suitable for this system.

(b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.

(c) Write a Java program using HashSet and HashMap with Generics to:

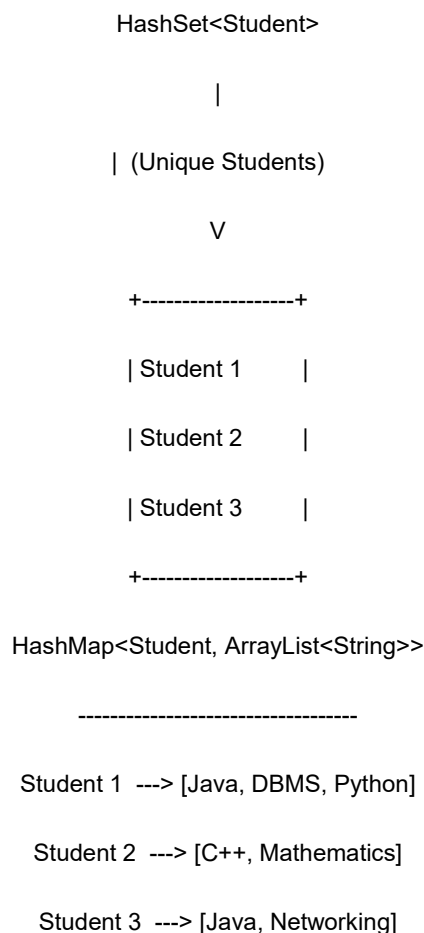
- Add students
- Map students to courses
- Display data using Iterator

Answer:

(a) Three Collection interfaces suitable for this system

1. **Set** – Stores unique student objects (no duplicates).
2. **List** – Stores courses in an ordered manner (duplicates allowed if needed).
3. **Map** – Stores the relationship between a student and the courses they have enrolled in.

(b) Collection Structure



(C.)Input:

```
import java.util.*;

public class StudentCourse {
    public static void main(String[] args) {

        // HashSet to store unique students
        HashSet<String> students = new HashSet<>();

        students.add("Alice");
        students.add("Bob");
        students.add("Charlie");

        // HashMap to map students with courses
        HashMap<String, String> courses = new HashMap<>();

        courses.put("Alice", "Java");
        courses.put("Bob", "Python");
        courses.put("Charlie", "C++");

        // Display using Iterator
        Iterator<String> it = students.iterator();

        while (it.hasNext()) {
            String student = it.next();
            System.out.println("Student : " + student);
            System.out.println("Course : " + courses.get(student));
            System.out.println();
        }
    }
}
```

Output:

Student : Alice
Course : Java

Student : Bob
Course : Python

Student : Charlie
Course : C++

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

- (a) List three Collection classes applicable for cart management.
- (b) Represent how cart items are stored using a List-based structure.
- (c) Write a Java program using ArrayList and Iterator to:
 - Add items
 - Remove an item
 - Display all items

(a) Three Collection classes applicable for cart management

1. **ArrayList** – Stores cart items in insertion order.
2. **LinkedList** – Efficient for adding and removing items.
3. **Vector** – Similar to ArrayList but synchronized (thread-safe).

(b) List-Based Structure

ArrayList<String>

```
-----  
| Laptop      |  
| Mouse      |  
| Keyboard    |  
| Headphones  |  
-----
```

Input:

```
import java.util.*;
```

```
public class ShoppingCart {  
    public static void main(String[] args) {
```

```
        // Create ArrayList  
        ArrayList<String> cart = new ArrayList<>();
```

```
        // Add items  
        cart.add("Laptop");  
        cart.add("Mouse");  
        cart.add("Keyboard");
```

```
        // Remove an item  
        cart.remove("Mouse");
```

```
        // Display items using Iterator  
        Iterator<String> it = cart.iterator();
```

```
        System.out.println("Items in Cart:");  
        while (it.hasNext()) {  
            System.out.println(it.next());  
        }
```

```
    }  
}
```

Output:

```
Items in cart:  
Laptop  
Keyboard
```

Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

(a) Name three Map implementations in Java.

(b) Design a schema-like representation using Map for storing ISBN and book details.

(c) Write a Java program using HashMap to:

- Insert book details
- Retrieve a book
- Display all books

(a) Three Map implementations in Java

1. **HashMap** – Stores data in key-value pairs (fast retrieval).
2. **LinkedHashMap** – Maintains insertion order.
3. **TreeMap** – Stores keys in sorted order.

(b) Map Representation

HashMap<String, String>

```
-----  
ISBN      Book Name  
-----  
101       Java Programming  
102       Data Structures  
103       Python Basics  
-----
```

Input:

```
import java.util.*;  
  
public class Library {  
  
    public static void main(String[] args) {  
  
        // Create HashMap  
  
        HashMap<String, String> books = new HashMap<>();  
  
        // Insert book details  
  
        books.put("101", "Java Programming");  
  
        books.put("102", "Data Structures");  
  
        books.put("103", "Python Basics");  
    }  
}
```

```

// Retrieve a book

System.out.println("Book with ISBN 102: " + books.get("102"));

// Display all books

System.out.println("\nAll Books:");

for (Map.Entry<String, String> entry : books.entrySet()) {

    System.out.println("ISBN: " + entry.getKey() +

        " Book: " + entry.getValue());

}

}

```

Output:

Book with ISBN 102: Data Structures

All Books:

ISBN: 101 Book: Java Programming

ISBN: 102 Book: Data Structures

ISBN: 103 Book: Python Basics

Q4: Employee Data Processing (10 Marks)

Suppose that a company processes employee records and filters employees based on salary.

- (a) List three advantages of Generics in Collections.
- (b) Represent how employee data is stored using List with Generics.
- (c) Write a Java program using Iterator to:
 - Store employee objects
 - Display employees with salary > 50,000

(a) Three advantages of Generics in Collections

1. **Type Safety** – Prevents storing the wrong type of objects.
2. **No Type Casting** – No need to convert objects manually.
3. **Code Reusability** – The same code works with different data types.

(b)Representation using List with Generics

ArrayList<Employee>

Name	Salary
Ravi	45000
Anu	60000
Kiran	75000

Input:

```
import java.util.*;
```

```
class Employee {
```

```
    String name;
```

```
    int salary;
```

```
    Employee(String name, int salary) {
```

```
        this.name = name;
```

```
        this.salary = salary;
```

```
    }
```

```
}
```

```
public class EmployeeData {
```

```
    public static void main(String[] args) {
```

```
        // Create List
```

```
        ArrayList<Employee> list = new ArrayList<>();
```

```
        // Store employee objects
```

```
        list.add(new Employee("Ravi", 45000));
```

```
        list.add(new Employee("Anu", 60000));
```

```
        list.add(new Employee("Kiran", 75000));
```

```
        // Display employees with salary > 50000
```

```
        Iterator<Employee> it = list.iterator();
```

```
        System.out.println("Employees with Salary > 50000:");
```

```
        while (it.hasNext()) {
```

```
            Employee e = it.next();
```

```
            if (e.salary > 50000) {
```

```
                System.out.println("Name: " + e.name +  
                                   " Salary: " + e.salary);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```


Ouput:

Employees with Salary > 50000:

Name: Anu Salary: 60000

Name: Kiran Salary: 75000

Q5: Online Voting System (10 Marks)

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

(a) Identify three suitable Collection types for this system.

(b) Design a structure using Set and Map for voters and vote counting.

(c) Write a Java program using HashSet and HashMap to:

- Record votes
- Prevent duplicate voting
- Display results

(a) Three suitable Collection types for this system

1. **HashSet** – Stores unique voters (prevents duplicate voting).
2. **HashMap** – Stores candidates and their vote counts.
3. **TreeMap** – Stores vote counts in sorted order (optional).

(b)Strucutre using Set and Map

HashSet<String> (Voters)

Voter1

Voter2

Voter3

HashMap<String, Integer> (Vote Count)

Candidate	Votes
-----------	-------

Alice	2
-------	---

Bob	1
-----	---

Charlie	0
---------	---

Input:

```
import java.util.*;

public class OnlineVoting {

    public static void main(String[] args) {

        // HashSet to store voters

        HashSet<String> voters = new HashSet<>();

        // HashMap to store vote count

        HashMap<String, Integer> votes = new HashMap<>();

        // Initialize candidates

        votes.put("Alice", 0);

        votes.put("Bob", 0);

        // Voter1 votes for Alice

        if (voters.add("Voter1")) {

            votes.put("Alice", votes.get("Alice") + 1);

        }

        // Voter2 votes for Bob

        if (voters.add("Voter2")) {

            votes.put("Bob", votes.get("Bob") + 1);

        }

        // Voter1 tries to vote again

        if (voters.add("Voter1")) {

            votes.put("Alice", votes.get("Alice") + 1);

        } else {

            System.out.println("Duplicate vote not allowed!");

        }

        // Display results

        System.out.println("\nVoting Results:");
```

```
    for (Map.Entry<String, Integer> entry : votes.entrySet()) {  
        System.out.println(entry.getKey() + " : " + entry.getValue());  
    }  
}  
}
```

Ouput:

Duplicate vote not allowed!

Voting Results:

Alice : 1

Bob : 1